



THE LAB CODER

Advanced Notes Part 3 (update 7 Nov 2025)

Notes: <http://tinyurl.com/TheLabAdvanced3>

OBJECT-ORIENTED PROGRAMMING

Object-oriented programming (OOP) is a programming paradigm based on the concept of "objects", which can contain data and code: data in the form of fields (often known as attributes or properties), and code, in the form of procedures (often known as methods).

- Wikipedia

Content

Page

4

[3048] Class

16

[3049] Class Variable

20

[3050] Static Method

24

[3051] Class Composition and Inheritance

32

[3052] Class Polymorphism

37

Access Modifier

40

Conclusion

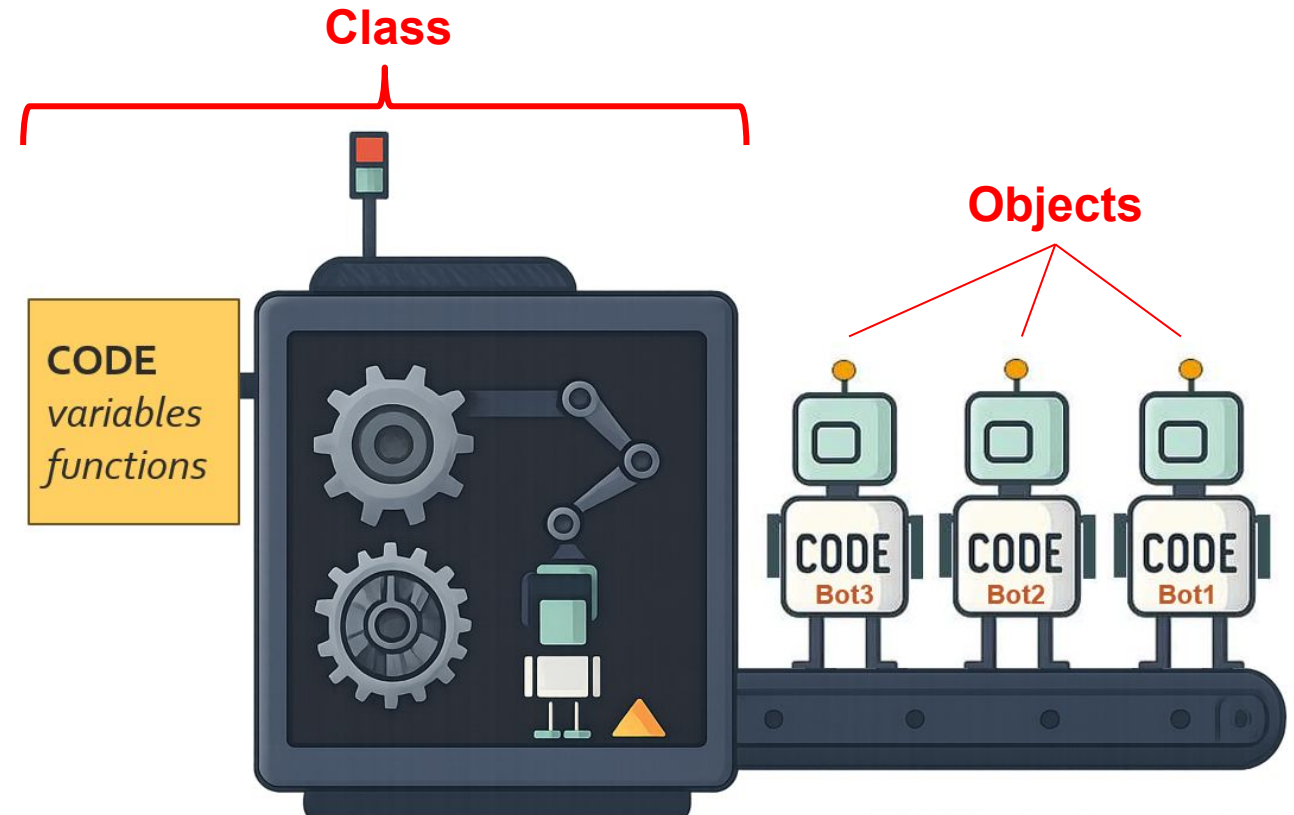
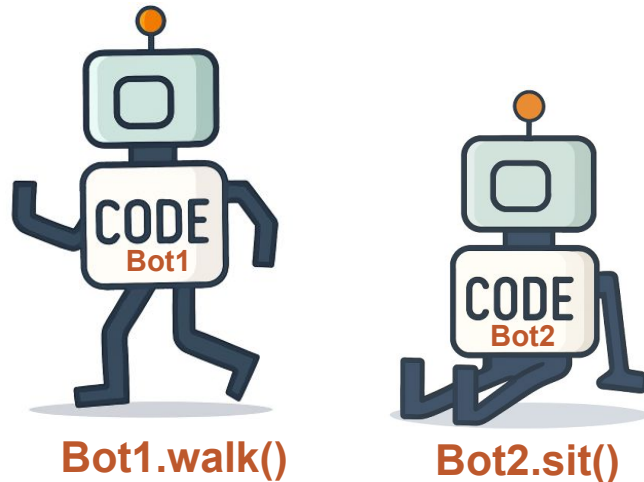
Click on titles

CLASS

Object-Oriented Programming

Class and **Object** are the common terms you will hear in OOP concept.

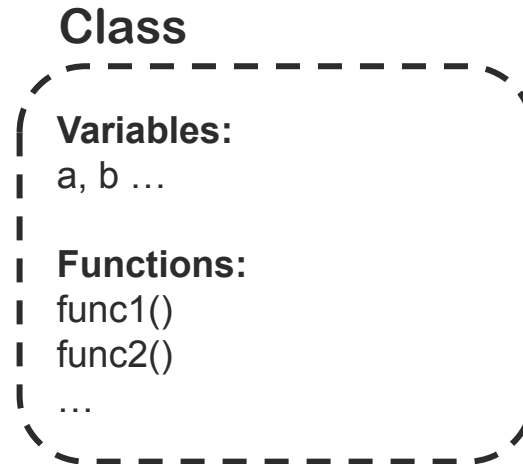
Think of a **class** like a machine that assembles a robot and the assembled robot is an **object**. Each object has a copy of the code.



Each robot can activate its function through its unique name (variable)

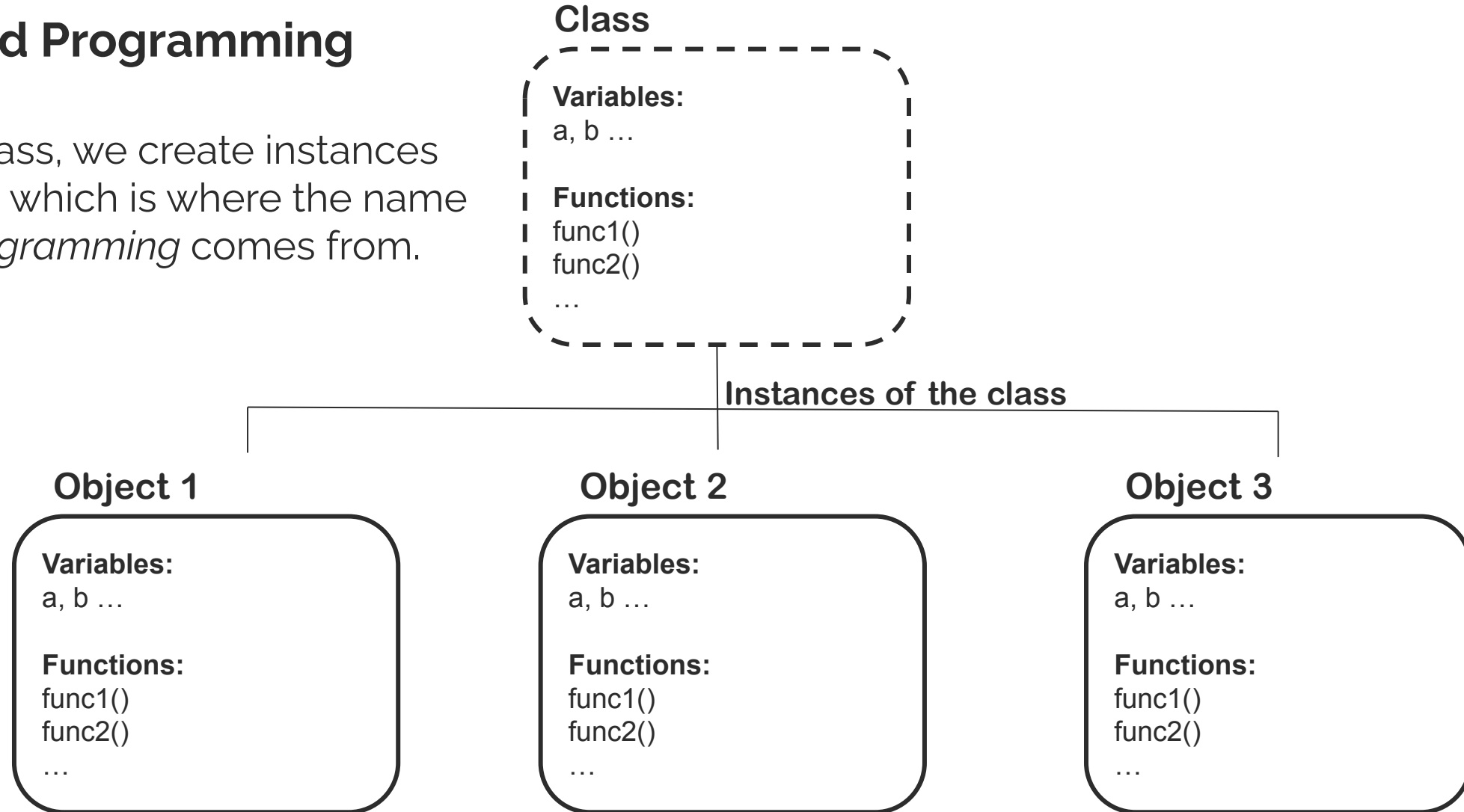
Object-Oriented Programming

A *class* is like a template with its own variables and related functions.



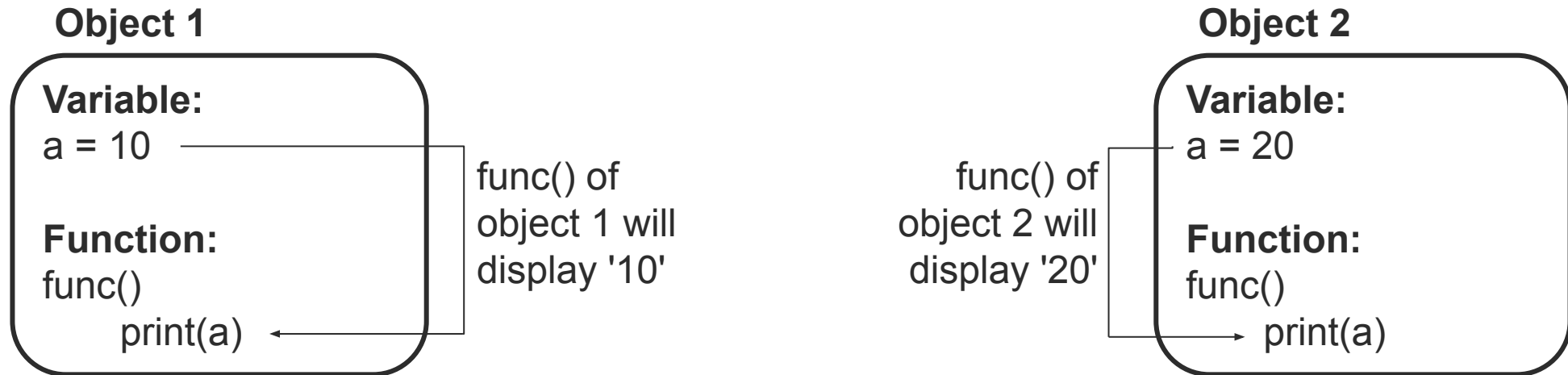
Object-Oriented Programming

In order to use a class, we create instances of it called **objects**, which is where the name *object-oriented programming* comes from.



Object-Oriented Programming

Each object will have its *own set* of variables and functions. Although they have the same names, they are separate for each object.



Defining a Class (Example #1)

class Hero:

__init__ is the object **constructor**. '***__***' is a double underscore
it must have ***self*** as the input parameter. Other input parameters are optional.
constructor is automatically called every time you create an object from a class

def *__init__* (self):

object variables, each object (instance) of a class will have its own copy
initialise to a default value as appropriate

self.name = "unknown"

other function that you create (function is also called *method*)

it must also have at least ***self*** as the input argument

def display (self):

print("Hero is" , self.name)

Car class

Object variable:
name

Function:
display()

Creating objects from a Class

Create an object via ***objectName = className()***

Access an object variable or method via ***objectName.variable*** and ***objectName.method()*** respectively.

instanting object from a class

using the class definition in previous slide

hero_object = **Hero()**

< Instanting an object from the class
 __init__() is called when an object is instanced

calling object function

hero_object.**display()**

< Output: *Hero is unknown*
 hero_object is the "*self*" argument of the function.
 Hence, "*self.name*" in the function refers to its variable.

accessing object variable/method

hero_object.**name** = "Thor"

hero_object.**display()**

< Sets the object variable to a value
< Output: *Hero is Thor*

adding new variables to object

hero_object.**power** = "super strength"

< This statement ***adds a new variable*** 'power' in hero_object



Defining a Class (Example #2)

class Hero:

constructor can have input arguments, usually for initialising upon creation

def __init__ (self, n, h):

self.name = n

self.health = h

function to display info

def display (self):

 print("Hero is" , **self.name** , "and health is", **self.health**)

function to reduce health value by the argument value

def injure (self, damage):

self.health = **self.health** - damage

Car class

Object variable:

 name

 health

Function:

 display()

 injure()



PYTHON CODING

CLASS

```
# Instanting object from a class using the class definition in previous slide
# Provide the 2 values to the input arguments in __init__()
hero1 = Hero("Black Widow", 100)    < Instanting an object from the class
hero2 = Hero("Iron Man", 100)       < Instanting another object from the class
```

```
# calling objects' display method
hero1.display()          < Output: Hero is Black Widow and health is 100
hero2.display()          < Output: Hero is Iron Man and health is 100
```

```
# calling injure method of the first object
hero1.injure(5)
```

```
# calling objects' display method
hero1.display()          < Output: Hero is Black Widow and health is 95
hero2.display()          < Output: Hero is Iron Man and health is 100
```



Passing Objects as Arguments

Objects can also be **passed as arguments** or **returned** as results in class methods.

```
class Hero:

    ...

    # example of taking in an object as input argument
    def compare(self, object):
        if self.power > object.power:
            print( self.name , "is more powerful than" , object.name)

    ...
```

Creating a list of objects

METHOD 1: by assignment

```
herolist = [ Hero('Wonder Woman' , 100) , Hero('Superman' , 100) , Hero('Black Panther' , 100) ]
```

METHOD 2: using for-loop (same values for all objects)

```
herolist = [ Hero( 'unknown' , 100 ) for i in range(3) ]
```

METHOD 3: appending each object

```
herolist = [ ]  
for i in range(3):  
    name = input("Enter name: ")  
    herolist.append( Hero( name , 100 ) )
```

The list operations are the same as normal list. Below shows an example of how to access a variable or call a method of an object in the list,

accessing object variables and functions (functions are also called *methods*)

```
herolist[0].health = 120    < changing variable of Wonder Woman  
herolist[2].display()      < calling the method of Black Panther
```



Layout

In a program file, arrange your codes in the layout shown on the right.

program file

library imports
import ...

all class definition
class 1
class 2
class 3
...

main
Execution of your
codes starts here

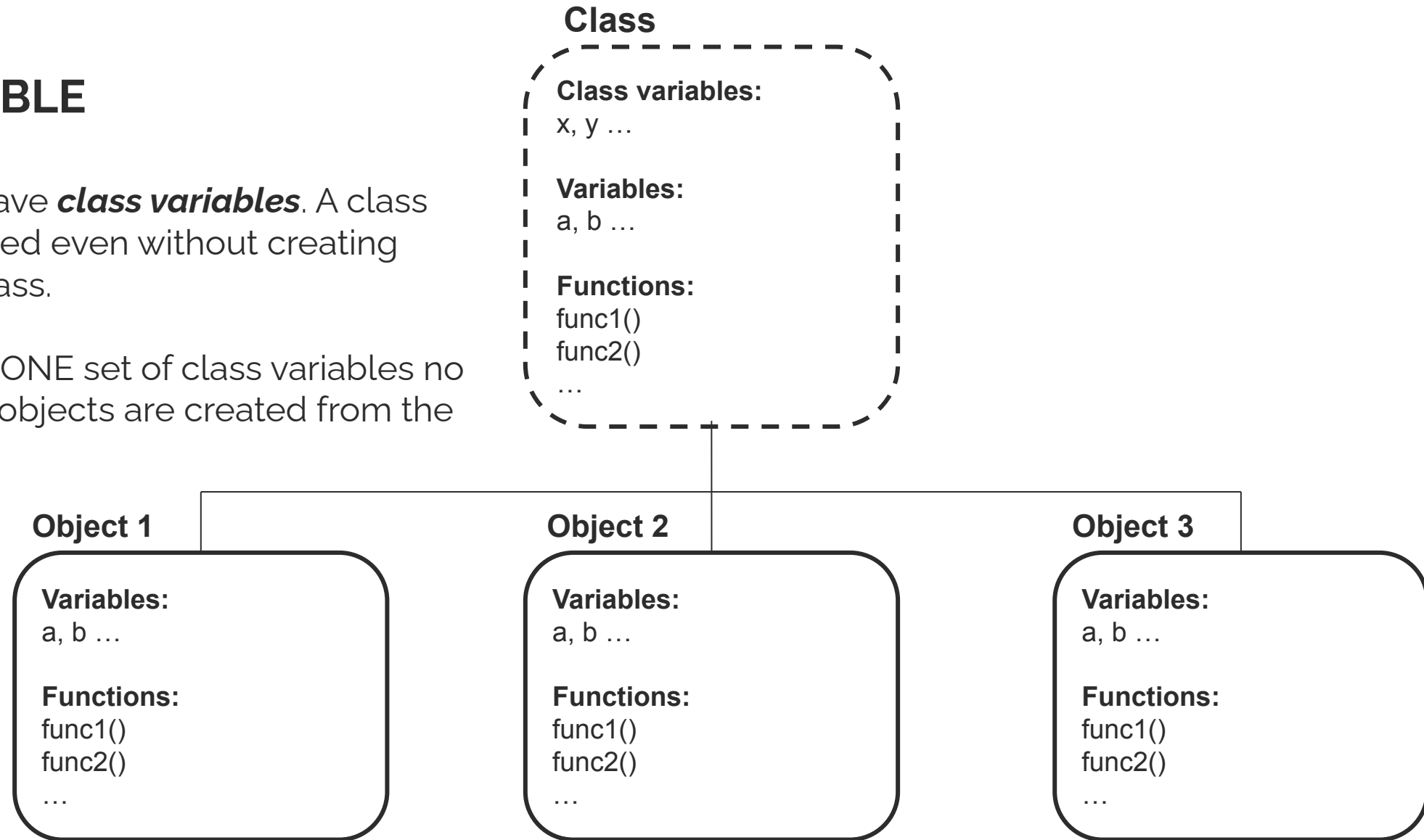
CLASS

CLASS VARIABLE

CLASS VARIABLE

A class can also have **class variables**. A class variable can be used even without creating object from the class.

There will only be ONE set of class variables no matter how many objects are created from the class.



Defining a Class with Class Variable

class Car:

```
# class variables
total_car = 0      < class variables are declared here

# object constructor
def __init__ (self, b, c):
    # object variables
    self.brand = b
    self.colour = c
    # (example) increase by 1 each time an object is created
    Car.total_car = Car.total_car + 1    < note the syntax of accessing
                                         the class variable using class name

# functions (also called methods)
def display_car(self):
    print( "Car brand is" , self.brand , "and colour is" , self.colour )

def display_total(self):
    print( "Total cars: " , Car.total_car )
```

Car Class

Class variable:
total_car

Object variables:
brand, colour

Functions:
display_car()
display_total()



Example

accessing class variable

```
print("Total cars:", Car.total_car)
```

< Output; *Total cars: 0*

You can use the class variable even without any object

instanting a class (create objects)

```
car1 = Car( 'Toyota' , 'grey' )
```

< Instanting object car1

```
car2 = Car( 'Chevrolet' , 'blue' )
```

< Instanting object car2

accessing class variable (all of them are accessing the same class variable)

```
car1.display_total()
```

< Output: *Total cars: 2*

```
car2.display_total()
```

< Output: *Total cars: 2*

```
print("Total cars:", Car.total_car)
```

< Output: *Total cars: 2*

WARNING: this adds a new variable to object

```
car1.total_car = 1
```

< This statement **creates a new variable** 'total_car' in car1 instead.
It is *NOT* accessing the Class variable.

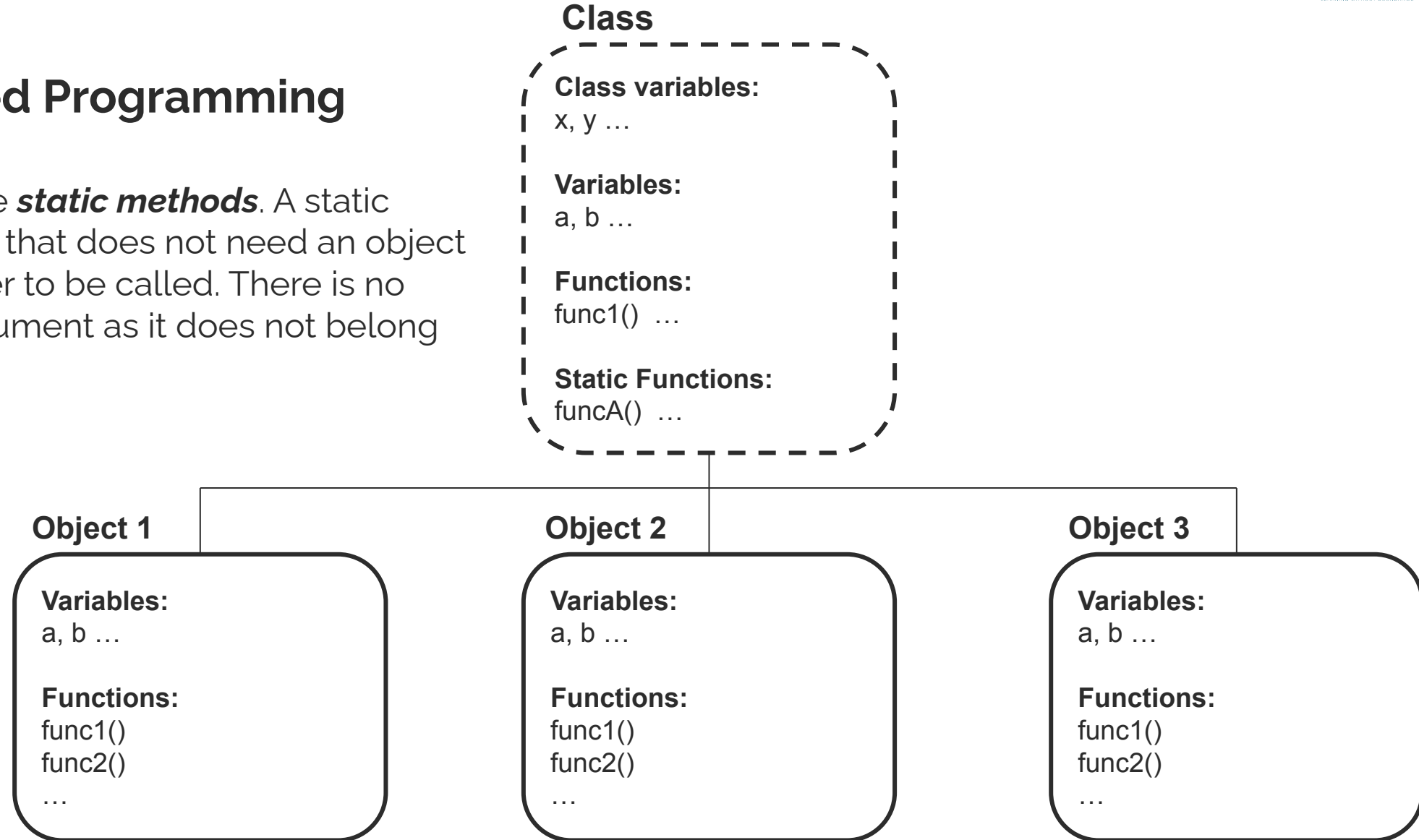


CLASS

STATIC METHOD

Object-Oriented Programming

A class can also have ***static methods***. A static method is a function that does not need an object to be created in order to be called. There is no 'self' in the input argument as it does not belong to any child object.



Defining a Class with Static Method

class Car:

class variable

total_car = 0

object constructor

def __init__(self, b, c):

object variables

self.brand = b

self.colour = c

Car.total_car = Car.total_car + 1

function (also called method)

def display_car(self):

print("Car brand is" , self.brand , "and colour is" , self.colour)

static method

@staticmethod < use '@staticmethod' to indicate a static method

def display_total(): < note that you do not need to put 'self' in parameter of static method

print("Total cars: " , Car.total_car)

Car class

Class variable:
total_car

Object variables:
brand, colour

Function:
display_car()

Static method:
display_total()



Example

Calling static method using class name

Car.display_total() < Output: *Total cars: 0*

car1 = Car('Toyota' , 'grey')

Car.display_total() < Output: *Total cars: 1*

car1.display_total() < Output: *Total cars: 1*

You can also access the static method through an object.

The result is the same but using class name to call is a better practice for code clarity



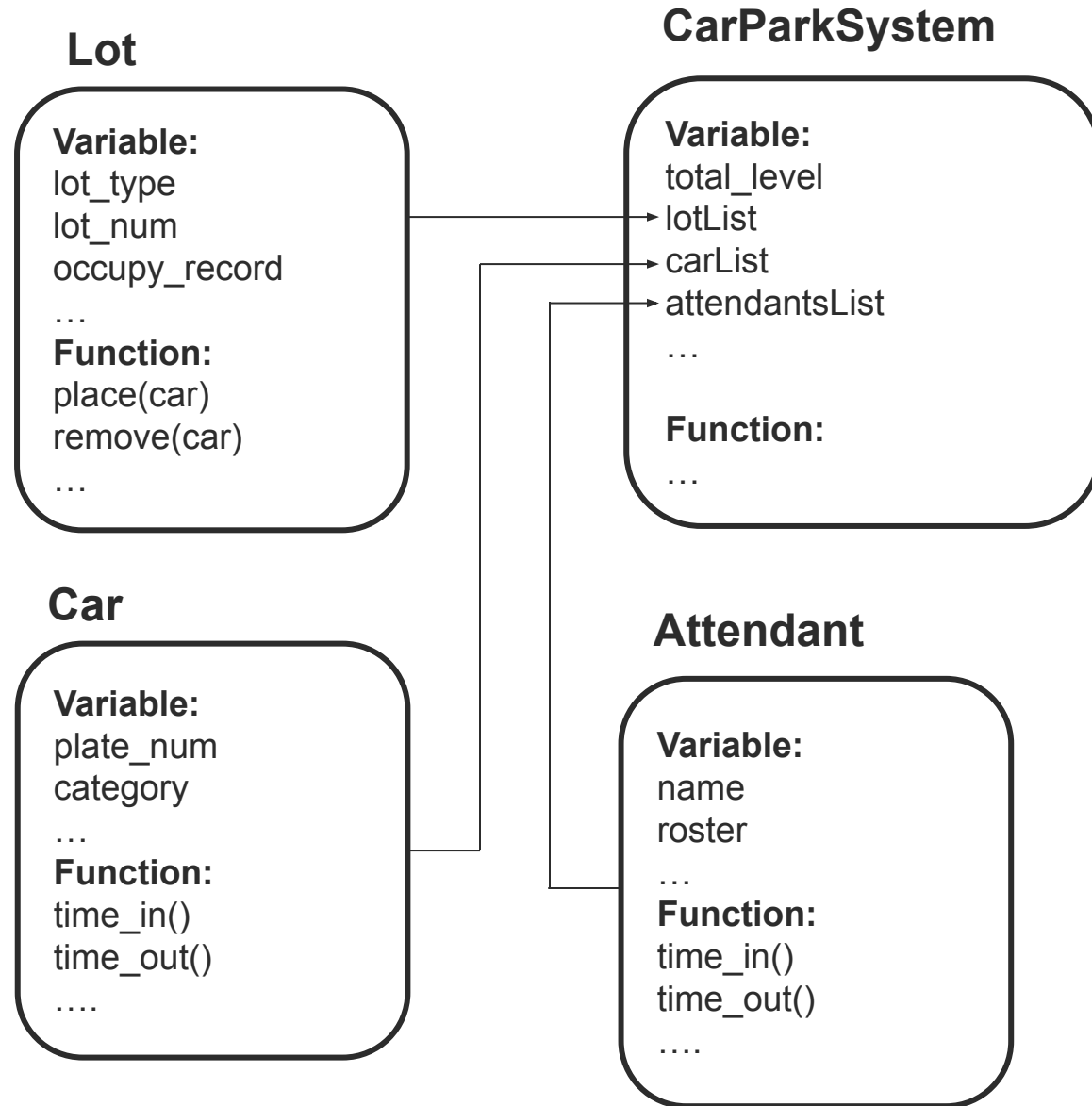
CLASS

COMPOSITION AND INHERITANCE

Class Composition

In OOP, we often design our programs with multiple classes that interact with one another.

You can create ***objects of other classes*** in another class.



Class Composition

Objects from different classes can also be **passed as parameters** or **returned** as results in class methods.

```
class HeroTypeA:

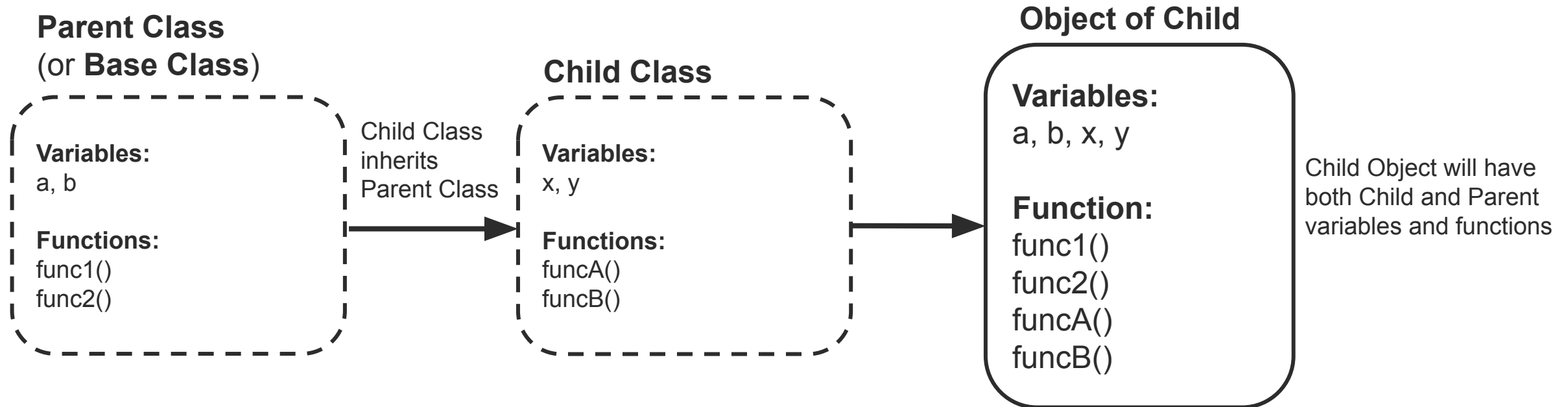
    ...

    # example of taking in an object as input argument
    def compare(self, objectTypeB):
        if self.power > objectTypeB.power:
            print( self.name , "is more powerful than" , objectTypeB.name)

    ...
```

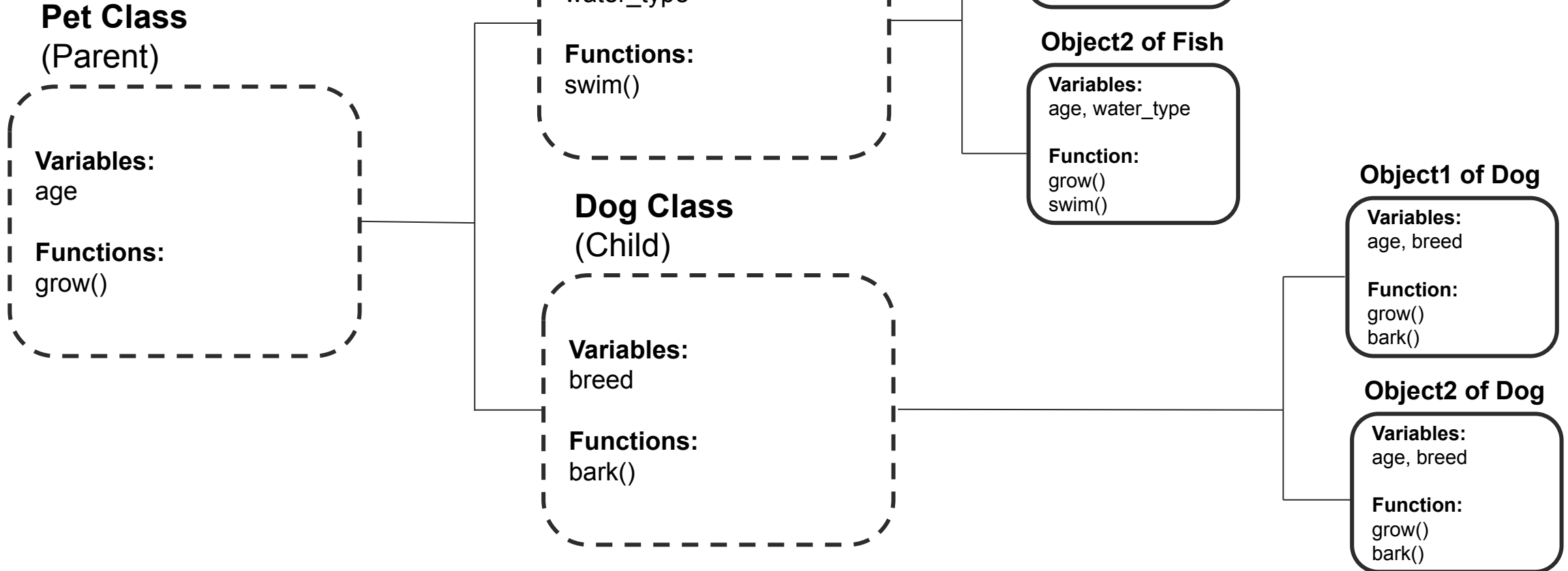
Class Inheritance

A class can inherit another class to include its properties (variables and functions).



Class Inheritance

Example



Defining a Class with Inheritance

Child class inheriting Parent class
class Dog(**Animal**):

Parent (Base) class
class Animal:

object constructor
def __init__(self, a):
 # object variable
 self.age = a

method
def display_age(self):
 print("Age is" , self.age)

class variables
total = 0

object constructor
def __init__(self, a, b):
 # call constructor of parent class
 Animal.__init__(self, a)
 # object variable
 self.breed = b

method
def bark(self):
 print(self.breed, "barks.")

static method
@staticmethod
def display_total():
 print("Total is" , Dog.total)

Dog class

Class variable:
total

Object variables:
age, breed

Functions:
display_age()
bark()

Static method
display_total()



Example

instanting the child class

```
dog1 = Dog( 3, 'Poodle' )    < Instancing object dog1  
dog2 = Dog( 1, 'Chow chow' ) < Instancing object dog2
```

accessing object variable

```
dog1.age = dog1.age + 1
```

accessing class variable

```
Dog.total = 2
```

calling methods

```
dog1.bark()                < Output: Poodle barks.
```

```
dog1.display_age()         < Output: Age is 4
```

```
dog2.bark()                < Output: Chow chow barks.
```

```
dog2.display_age()         < Output: Age is 1
```

```
Dog.display_total()        < Output: Total is 2
```



Class Inheritance

A child class can also inherit multiple parent classes.

If there are identical variables or functions, they will be overwritten by the latest initialisation.

Lot Class (Parent)

Variables:

lot_num

Functions:

place_car()

remove_car()

Attendant Class (Parent)

Variables:

attendant_name

Functions:

clock_in()

clock_out()

Carpark Class (Child)

Variables:

lot_num

attendant_name

Functions:

place_car()

remove_car()

clock_in()

clock_out()

declaration format

class Carpark(**Lot, Attendant**):

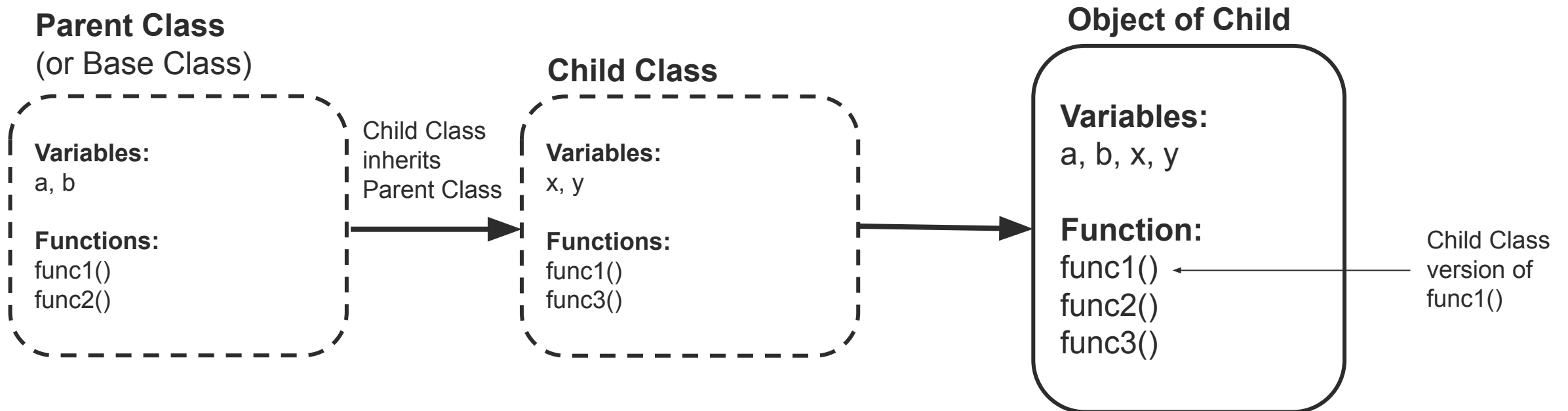
CLASS

POLYMORPHISM

(METHOD OVERRIDING)

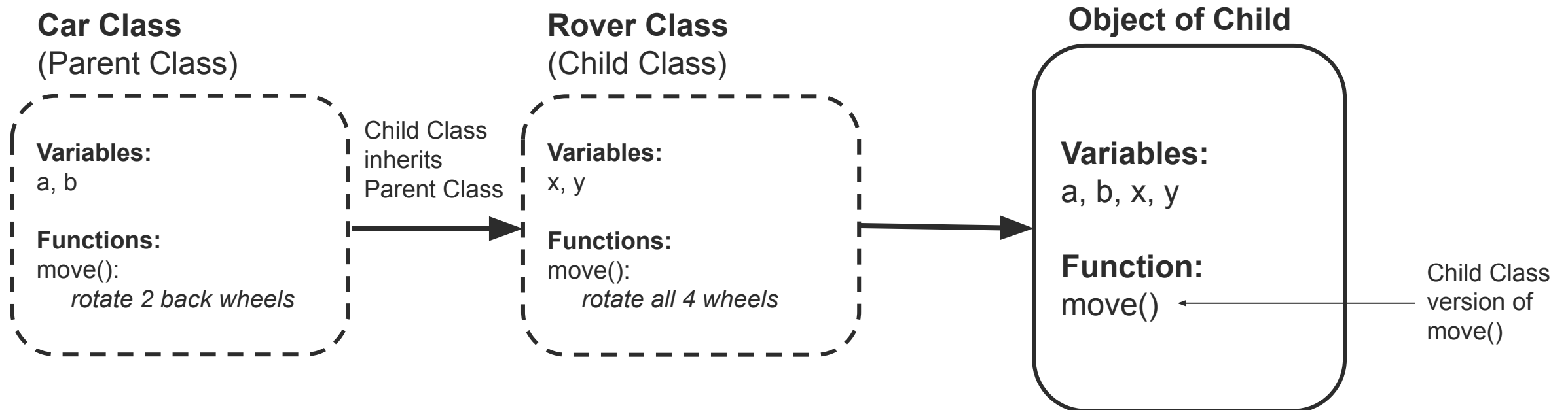
Class Polymorphism

A Child Class can override an existing function in its Parent Class.
This is called ***method overriding*** or ***polymorphism***.



Class Polymorphism

Example



Defining a Class with Polymorphism

Parent (Base) class
class Car:

object constructor
def __init__(self, b):
 # object variable
 self.brand = b

method
def move(self):
 print("Turning 2 wheels")

Overridden in
Child Class

Child class inheriting Parent class
class FourWheelDrive(Car):

object constructor
def __init__(self, b):
 # call parent class constructor
 # can also use **Car.__init__(self, b)**
 super().__init__(b)

method overriding
def move(self):
 print("Turning 4 wheels")

FourWheelDrive class

Object variable:
brand

Function:
move()



Example

instantcing the parent class

```
car1 = Car( 'Toyota' )
```

instantcing the child class

```
car2 = FourWheelDrive( 'Land Rover' )
```

calling object function

```
car1.move()      < Output: Turning 2 wheels
```

```
car2.move()      < Output: Turning 4 wheels
```

ACCESS MODIFIER

This topic is for further reading to understand more about OOP in Python.

Access modifiers are used to restrict access to certain variables and methods of a class in order to prevent unauthorised access and exploitation. A Class in Python has three types of access modifiers:

- Public Access Modifier (default)
- Protected Access Modifier (indicated with a preceding underscore '_')
- Private Access Modifier (indicated with 2 preceding underscores '__')

Public Access Modifier

By default all variables and methods are public, meaning they can be accessed anywhere in the program.

Protected Access Modifier

Protected variables and methods of a class can only be accessed with the class and its child class (inherited). Simply precede a variable or method name with an underscore '_' to make it protected.

For examples:

```
self._myProtectedVariable  
def _MyProtectedFunction(self)
```

Private Access Modifier

Private variables and methods of a class can only be accessed within the class. Precede a variable or method name with two underscores '__' to make it private.

For examples:

```
self.__myPrivateVariable  
def __MyPrivateFunction(self)
```

However, note that Python only provides a 'weak' protection with preceding underscores. Be it protected or private, the members can still be accessed, just not in the usual object calling way (e.g. *object.__privateFunc()* will raise an error but *object._classname__privateFunc()* is valid!).



CONCLUSION

You have come to the end of The Lab Advanced Module. By now, you should have acquired the ability to create simple applications independently. You may be interested to explore deeper into specific areas such as game, web development, AI, etc. Below are suggestions to some libraries where you can start exploring.

- 1) Arcade / Pygame: game development with Python
- 2) Flask / Django: web developement with Python
- 3) TensorFlow / Numpy: AI or Datascience with Python,
- 4) Kivy: mobile apps
- 5) Blender: free 3D software which allows Python codes to be integrated for 3D animation, etc.